

EMPLOYEE TRACKING SYSTEM

Capstone Project Report

Technology Stack: Java 17, Spring Boot, Spring Security, Thymeleaf, MySQL, Bootstrap 5

Project Type: Full-Stack Web Application

Submission Date: 05 April 2026

INDEX

CHAPTER 1 :	
INTRODUCTION	3
CHAPTER 2 :	
LITERATURE REVIEW	5
CHAPTER 3 :	
METHODOLOGY	7
CHAPTER 4 :	
DESIGN AND IMPLEMENTATION	10
• System Flow Diagram	10
• Activity Lifecycle Diagram	11
• Use Case Diagram	12
• Data Flow Diagram (DFD)	13
• Class Diagram	14
• Sequence Diagram	15
CHAPTER 5 :	
TESTING	16
CHAPTER 6 :	
RESULT AND APPLICATIONS	18
CHAPTER 7 :	
CONCLUSION AND FUTURE SCOPE	24
CHAPTER 8 :	
REFERENCES AND BIBLIOGRAPHY	26

CHAPTER 1 : INTRODUCTION

1.1 Abstract

This capstone report presents the design and implementation of an **Employee Tracking System (ETS)**, a role-based web application developed using Spring Boot and MySQL. The system centralizes employee data management, attendance recording, leave processing, and user administration. It provides dedicated dashboards for administrators and employees, ensuring secure access and intuitive workflows.

The project follows layered architecture principles with clear separation across controller, service, repository, model, and view layers. Authentication and authorization are implemented with Spring Security and BCrypt password hashing, making the solution suitable for real-world organizational use.

1.2 Introduction

Organizations require a reliable digital system to maintain employee records and day-to-day HR operations. Manual or spreadsheet-based methods create inconsistencies, delay approvals, and limit reporting capabilities. ETS is designed to provide a compact, maintainable, and secure system that streamlines employee and attendance operations while improving visibility for administrators.

The system serves two types of users: administrators who manage all HR operations, and employees who interact with their own attendance and leave data. All interactions are protected by Spring Security, ensuring that no user can access resources beyond their role.

1.3 Problem Statement

Traditional employee record systems often suffer from scattered data, weak access control, and process delays in attendance and leave workflows. The project addresses these issues by developing a centralized portal with role-specific privileges, form validation, and controlled approval pipelines.

- Lack of centralized digital records for employee information.
- Manual and error-prone attendance tracking processes.
- No formal leave request and approval mechanism.
- Absence of role-based access separation for HR staff and employees.

1.4 Objectives

- Develop a secure web-based employee information management platform.
- Implement attendance check-in/check-out with historical records.
- Provide a leave request and approval workflow.

- Enable administrators to manage departments, users, and employee profiles.
- Ensure usability through a responsive UI and clean navigation.

1.5 Scope of the Project

The current scope includes employee CRUD, department management, attendance processing, leave application and approval, and user enable/disable operations. The solution supports two roles: ADMIN and EMPLOYEE. It targets educational and small-to-medium enterprise scenarios and can be extended for enterprise reporting.

CHAPTER 2 : LITERATURE REVIEW

A review of existing employee management and tracking systems reveals a range of solutions from enterprise platforms like SAP HCM and Oracle HRMS to open-source alternatives. This chapter examines the key findings from literature that informed the design decisions for ETS.

2.1 Existing Systems and Gaps

System / Study	Approach	Limitation
SAP HCM	Enterprise HR suite with full payroll integration	High cost, complex setup, not suitable for small organizations
Traditional Spreadsheet Systems	Excel-based manual records	No access control, prone to errors, no workflow automation
Open-source HRMS (OrangeHRM)	Modular HR management	Complex configuration, limited customization for developers
Custom PHP Applications	Lightweight web-based HR	Lack of security best practices, no standardized architecture

2.2 Relevant Technologies Studied

The following technologies were reviewed and selected based on industry adoption, documentation quality, and suitability for the project scale:

- **Spring Boot:** Provides auto-configuration, embedded server, and a production-ready foundation for Java web applications.
- **Spring Security:** A comprehensive security framework supporting authentication, authorization, and CSRF protection out of the box.
- **Spring Data JPA + Hibernate:** Simplifies data access through repository abstraction and ORM mapping, reducing boilerplate SQL.
- **Thymeleaf:** A server-side Java template engine with natural HTML syntax, ideal for MVC-based applications.
- **Bootstrap 5:** A front-end framework enabling rapid development of responsive, mobile-friendly UI components.
- **MySQL:** A widely-used open-source relational database with strong JPA/Hibernate integration.

2.3 Key Design Principles from Literature

- **Role-Based Access Control (RBAC):** Separation of admin and employee privileges as a core security principle.
- **Layered Architecture:** Clear separation of presentation, business logic, and data access layers for maintainability.
- **DTO Pattern:** Using Data Transfer Objects to decouple API boundaries from entity models.
- **Repository Pattern:** Abstracting data store operations behind repository interfaces for testability.

CHAPTER 3 : METHODOLOGY

This chapter describes the development approach, technology choices, system architecture, database schema, and security methodology used to build ETS.

3.1 Development Approach

The project follows an iterative development methodology. Each module (Authentication, Employee, Department, Attendance, Leave) was designed, implemented, and tested independently before integration. The MVC pattern with Spring Boot provides a clean separation of concerns throughout.

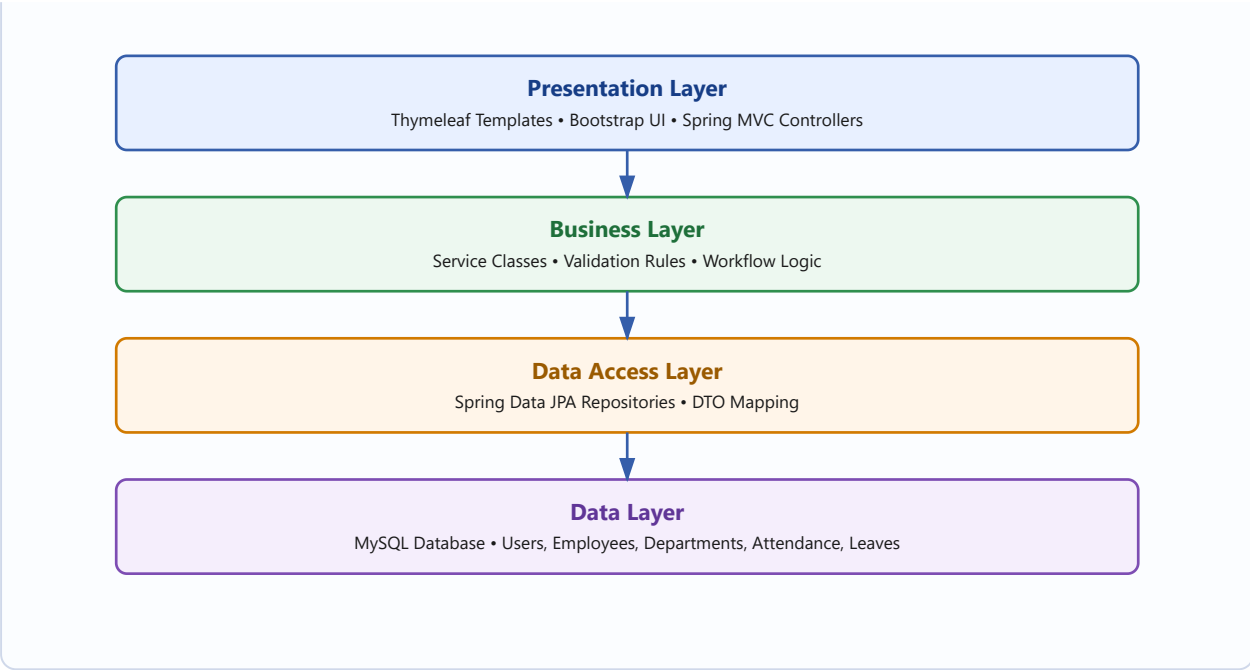
3.2 Technology Stack

Layer	Technology	Purpose
Language	Java 17+	Core application development
Framework	Spring Boot 4.0.3	Application runtime and dependency management
Security	Spring Security + BCrypt	Authentication, authorization, password protection
Persistence	Spring Data JPA + Hibernate	ORM and repository abstraction
Database	MySQL 8.0+	Relational data storage
Frontend	Thymeleaf + Bootstrap 5	Server-side rendering and responsive UI
Build Tool	Maven	Build, test, package lifecycle

3.3 System Architecture

ETS follows a four-layer architecture. Controllers process HTTP requests, services contain business logic, repositories handle data access, and entity models represent domain objects. DTOs are used for controlled data transfer across layers.





3.4 Database Design

The database is normalized around five main entities with foreign key relationships enforcing referential integrity.

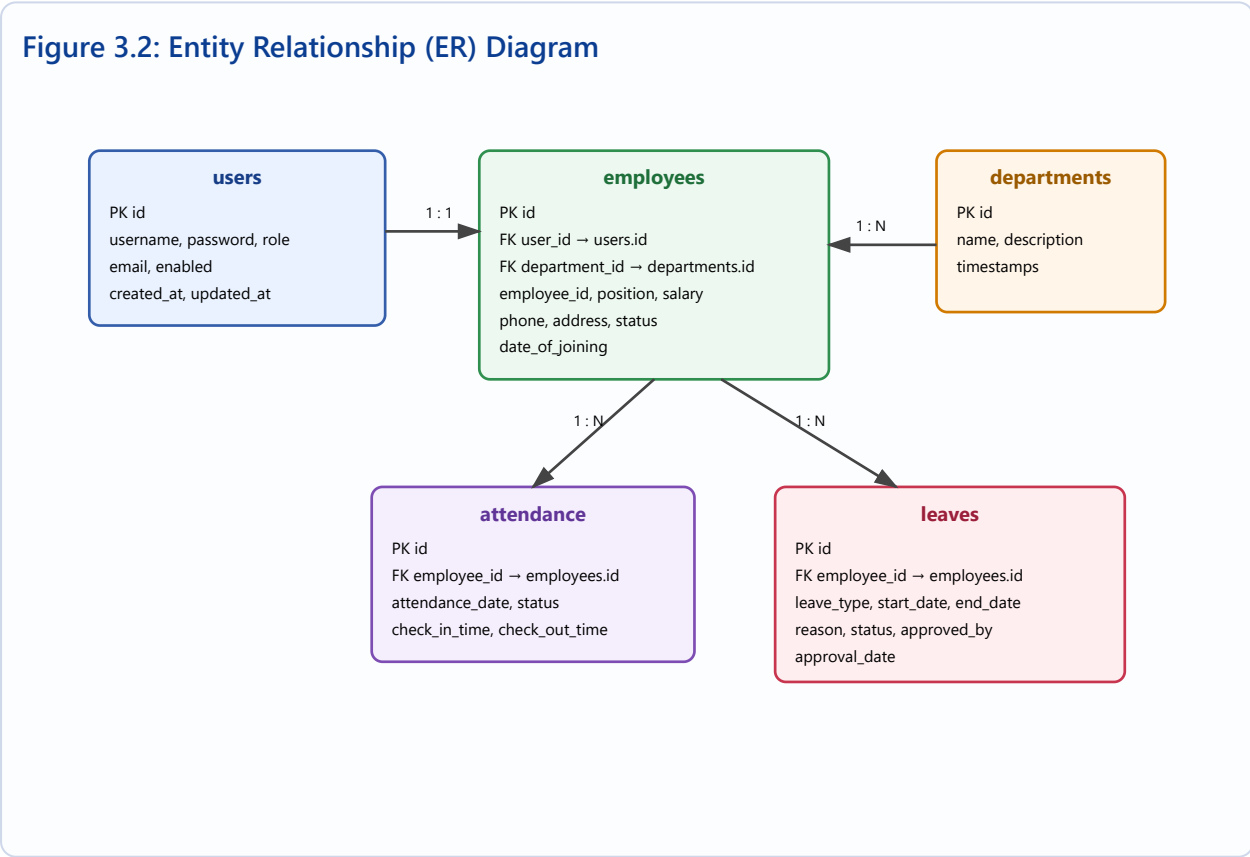
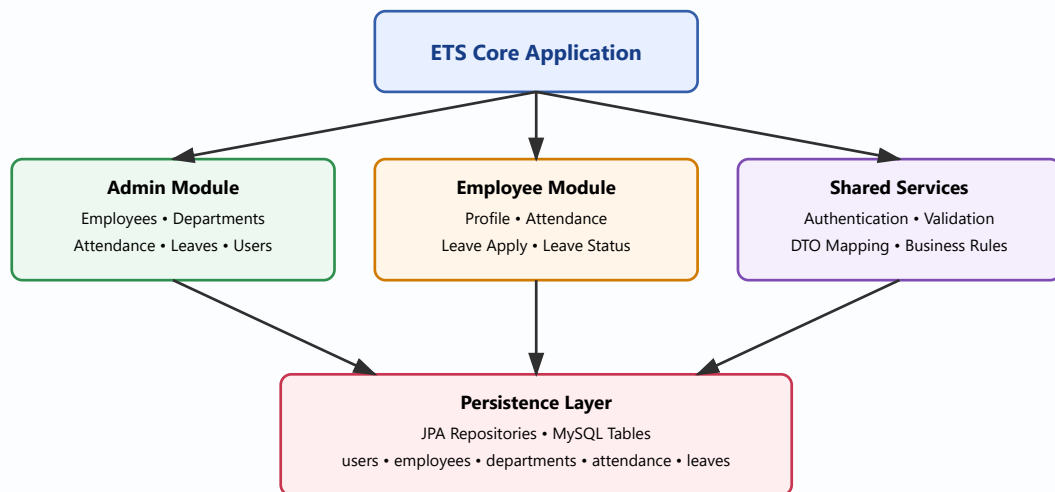


Table	Description	Key Relationships
users	Application authentication and role information	1-to-1 with employees

Table	Description	Key Relationships
employees	Employee personal and professional profile	Many-to-1 with departments
departments	Department metadata	Referenced by employees
attendance	Daily attendance log with check-in/check-out	Many-to-1 with employees
leaves	Leave applications and decision history	Many-to-1 with employees

3.5 Module Implementation

Figure 3.3: Functional Module Diagram

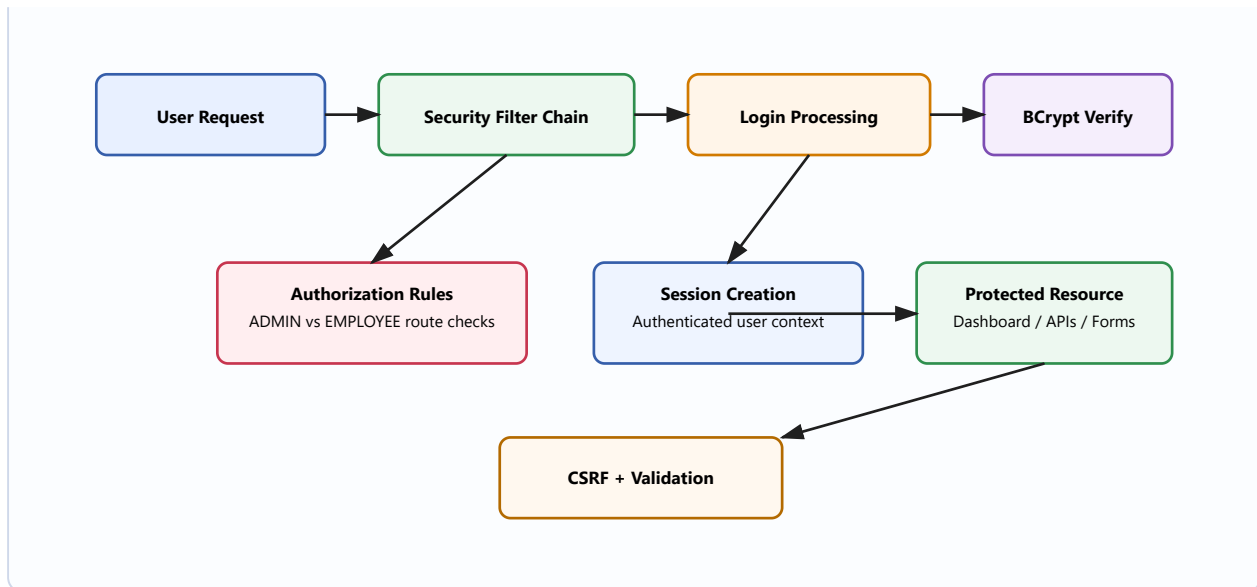


Admin Module: Dashboard, employee CRUD, department management, attendance monitoring, leave approvals, user account control.

Employee Module: Personal dashboard, attendance check-in/check-out, attendance history, leave application, leave status, profile view.

3.6 Security Design

Figure 3.4: Security Flow Diagram



- Role-based access control using ADMIN and EMPLOYEE authorities.
- Spring Security login/logout flow with protected routes.
- BCrypt hashing for credential storage.
- CSRF protection and secure form submissions.
- Validation constraints for input quality and integrity.

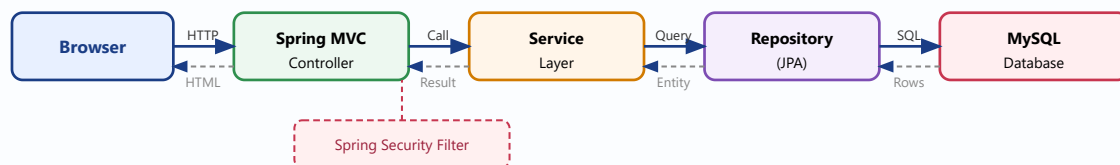
CHAPTER 4 : DESIGN AND IMPLEMENTATION

This chapter presents the detailed design artifacts of the Employee Tracking System, illustrating system flow, actor interactions, data movement, class relationships, and runtime message exchanges.

11.1 System Flow Diagram

The system flow shows the end-to-end request processing path from browser to database and back.

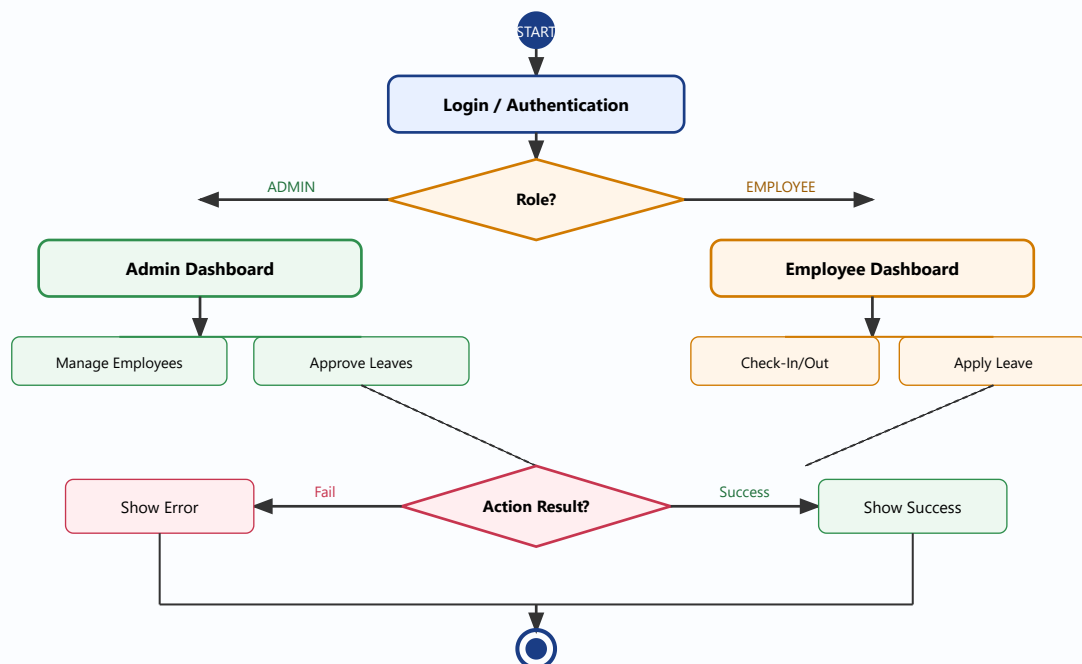
Figure 11.1: System Flow Diagram



11.2 Activity Lifecycle Diagram

The activity lifecycle diagram shows the main user workflows — login routing, attendance management, and leave processing — from start to completion.

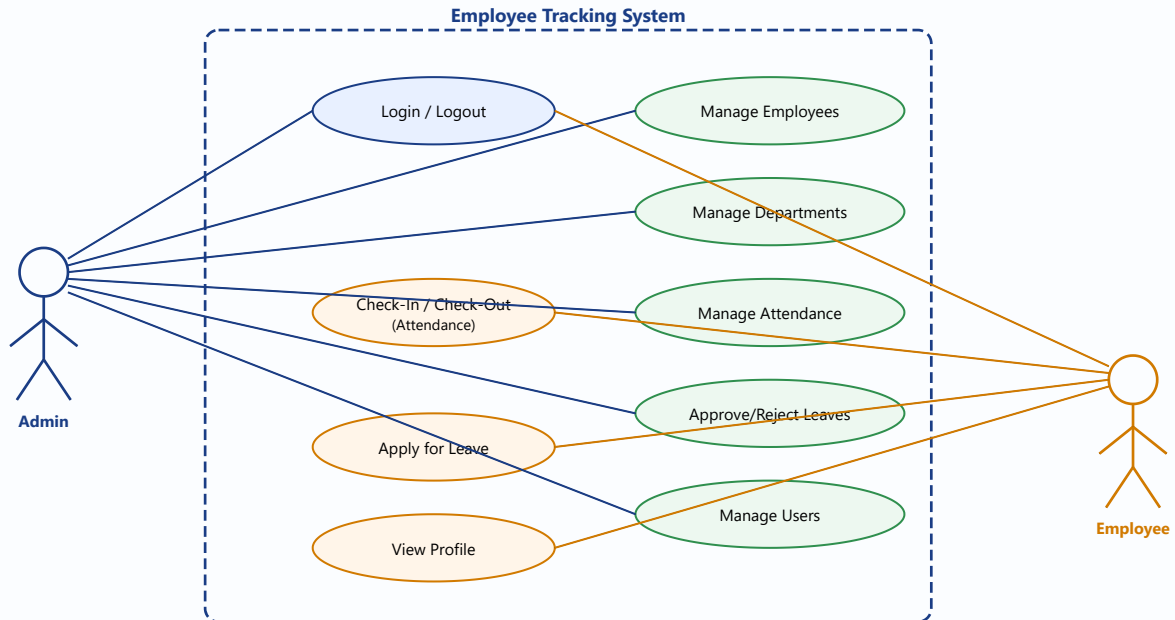
Figure 11.2: Activity Lifecycle Diagram



11.3 Use Case Diagram

The use case diagram identifies all actors and their interactions with the system's core functionalities.

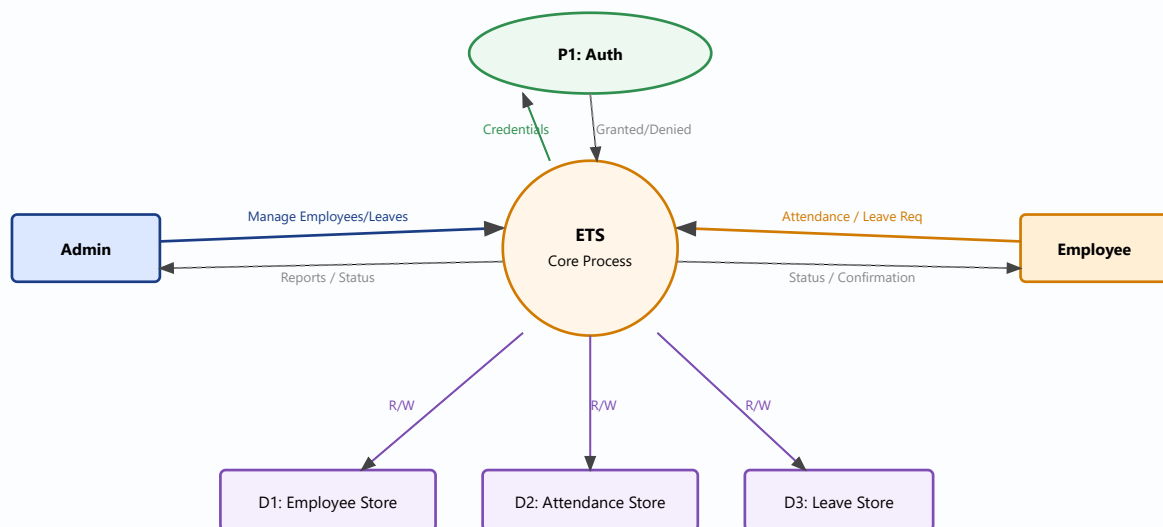
Figure 11.3: Use Case Diagram



11.4 Data Flow Diagram (DFD)

The DFD shows how data moves between external entities, processes, and data stores within the system.

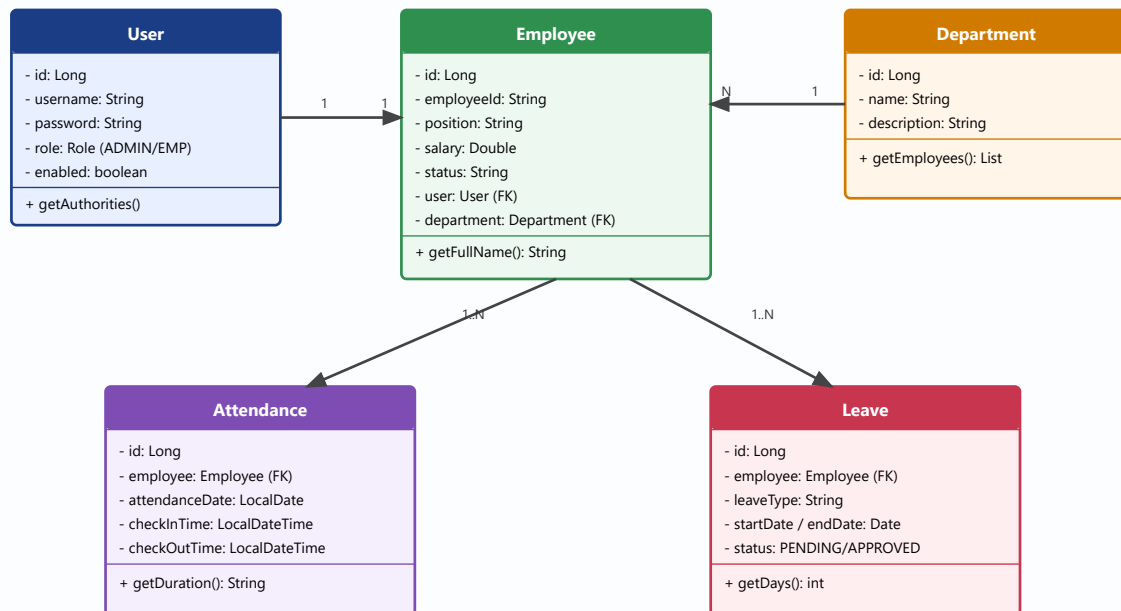
Figure 11.4: Level-1 Data Flow Diagram



11.5 Class Diagram

The class diagram shows the main domain entities, their attributes, and relationships as modeled in the JPA layer.

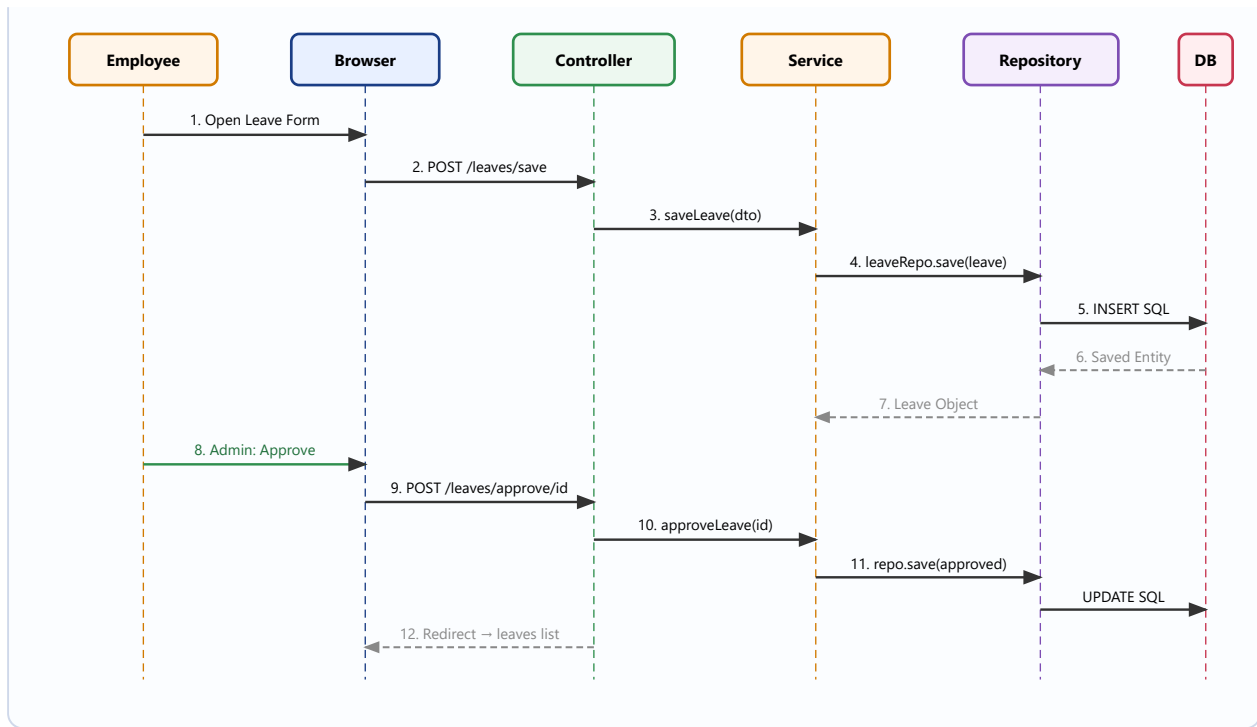
Figure 11.5: Class Diagram



11.6 Sequence Diagram

The sequence diagram shows the message exchange between components during the Leave Application flow.

Figure 11.6: Sequence Diagram — Leave Application & Approval Flow



CHAPTER 5 : TESTING

The project is validated through unit and integration test setup in Maven, route access verification by role, form validation checks, and end-to-end functional flow testing for attendance and leave operations.

Test Area	Verification	Status
Authentication	Valid and invalid login scenarios	Pass
Authorization	Route access based on role	Pass
Employee CRUD	Create/update/delete/list operations	Pass
Attendance	Check-in/check-out and history flow	Pass
Leave Workflow	Apply, approve, reject lifecycle	Pass

CHAPTER 6 : RESULT AND APPLICATIONS

6.1 Project Results and Outcomes

The final solution successfully delivers all targeted modules and provides a robust base for future enterprise extensions. The platform improves process consistency, decreases manual effort, and increases visibility over employee activities.

Key outcomes include complete CRUD functionality, role-segmented dashboards, secure authentication, clean UI implementation, and maintainable code organization.

6.2 Challenges and Solutions

Challenge	Approach	Outcome
Role-based page isolation	Centralized security configuration and URL policies	Stable permission boundaries
Data consistency across modules	Layered service logic and entity mapping	Predictable workflow behavior
Responsive UI for admin and employee screens	Bootstrap-based shared layout	Consistent cross-page UX
Validation and error feedback	Form binding + validation message handling	Improved user input quality

CHAPTER 7 : CONCLUSION AND FUTURE SCOPE

7.1 Future Enhancements

- Email and in-app notifications for leave decisions.
- Advanced analytics dashboard and PDF/Excel reporting.
- Payroll and biometric integration.
- REST API exposure for mobile clients.
- Cloud deployment with CI/CD and monitoring.

7.2 Conclusion

The Employee Tracking System capstone project demonstrates end-to-end full-stack development with secure architecture and practical business workflows. It satisfies major HR operational requirements and serves as a strong foundation for production-grade expansion.

CHAPTER 8 : REFERENCES AND BIBLIOGRAPHY

- [Spring Boot Official Documentation](#)
- [Spring Security Reference Guide](#)
- [Hibernate ORM Documentation](#)
- [Thymeleaf Documentation](#)
- [MySQL 8.0 Reference Manual](#)
- [Bootstrap 5 Documentation](#)

Prepared for academic capstone submission purposes.